

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 5 (PBL - Part 1)

Student Name: M.Vanisree

Roll No.: A24126552125

This record contains four experiments carried out under Task 5. Each experiment is documented with its aim, objective, theory, algorithm, program, code explanation, test cases with valid and invalid inputs, output screenshots and result.

Q. No.	Question	CO
1	Interactive Product Calculator: Create a webpage containing a product name, quantity and price. Using JavaScript functions and event listeners, calculate the total price dynamically whenever the quantity or price changes. Display the result without refreshing the page.	CO-1
2	Node.js File Management: Create a Node.js application that accepts a filename and content from the user and performs create/write, read, append and display of the final contents using the File System module.	CO-2
3	Express.js Student Server: Create a basic Express.js server with routes /, /students and /about. The /students route should return a list of at least five students, while /about should display information about the application.	CO-2
4	Explain the Document Object Model (DOM). Demonstrate how JavaScript can dynamically change the content, style and attributes of an HTML element.	CO-1

EXPERIMENT 1

1. TITLE

Interactive Product Calculator Using JavaScript Functions, Event Listeners and DOM Updates.

2. AIM

To create a webpage containing a product name, quantity and price, and to calculate and display the total price dynamically using JavaScript functions and event listeners whenever the quantity or price changes, without refreshing the page.

3. OBJECTIVE

The objectives of this experiment are:

- To design a simple input form using HTML and CSS with fields for product name, quantity and price.
- To select HTML elements from JavaScript using `document.getElementById()`.
- To read the values typed by the user with the `.value` property and convert them to numbers using `Number()`.
- To write a reusable function `calculateTotal()` that performs the multiplication and updates the page.
- To attach the function to the input event of the quantity and price fields using `addEventListener()`, so the total recalculates as the user types.
- To validate the entered data and display an error message for zero, negative or empty values instead of showing a wrong total.
- To update the page content through the DOM using the `textContent` property, so the result appears without a page reload.
- To format the calculated amount to two decimal places using `toFixed(2)`.

4. THEORY

This program is event-driven. The HTML page is loaded once, and after that every change the user makes to the quantity or price field fires an input event that runs a JavaScript function. Because the result is written back into the page through the DOM, the browser never reloads the document, and the total appears to update instantly as the user types.

A function is a reusable block of code. Here, the whole calculation and validation logic is placed inside a single function named `calculateTotal()`, and both input fields point to that same function. This avoids writing the same code twice and keeps the behaviour consistent.

The values returned by the `.value` property are always strings, even when the input is of type number. `Number()` converts these strings into numeric values so that multiplication produces a number and not a joined string. If the field is empty, `Number("")` returns 0, which is why the program also checks `price.value === ""` separately.

The key methods and concepts used in this program are:

- `document.getElementById("id")` returns the element on the page whose id attribute matches, so JavaScript can read from it or write into it.
- `element.value` reads the current text inside an input field at the moment the function runs.
- `Number(value)` converts the string read from the input into a numeric value so arithmetic can be performed.

- `addEventListener("input", calculateTotal)` runs `calculateTotal()` on every keystroke or spinner click in that field, unlike `"change"`, which waits until the field loses focus.
- `element.textContent` writes plain text into an element, replacing whatever was there before; this is how the total and the error message are displayed.
- `toFixed(2)` rounds the total to two decimal places and returns it as a string, which is suitable for displaying currency.
- The if condition `quantityValue <= 0 || priceValue < 0 || price.value === ""` rejects invalid data: a quantity of zero or less, a negative price, or a price left blank.
- `return` inside the if block stops the function immediately, so the invalid value is never multiplied or displayed.

The basic flow of the page is:

The user types or changes the quantity or the price → the input event fires → `calculateTotal()` runs → both values are read and converted to numbers → the values are validated → if they are invalid, an error message is shown and the total is reset to ₹0.00 → if they are valid, the error message is cleared, quantity is multiplied by price, and the result is written into the total paragraph with two decimal places.

5. ALGORITHM / PROCEDURE

- Create two files, `index.html` and `script.js`, and link `script.js` at the end of the body of `index.html`.
- In `index.html`, add a container `div` with a heading, a text input for the product name, a number input for the quantity with `min="1"` and `value="1"`, and a number input for the price with `min="0"` and `step="0.01"`.
- Add an empty paragraph with `id="error"` for the validation message and a paragraph with `id="total"` that initially shows Total: ₹0.00.
- Style the container, labels, inputs, total and error message with CSS, keeping the error text red.
- In `script.js`, select the product name, quantity, price, total and error elements using `document.getElementById()`.
- Define a function `calculateTotal()` that reads `quantity.value` and `price.value` and converts both with `Number()`.
- Inside the function, check whether the quantity is less than or equal to zero, the price is negative, or the price field is empty.
- If any of these conditions is true, set the error paragraph to "Please enter a valid quantity and price.", reset the total to Total: ₹0.00 and return from the function.
- If the data is valid, clear the error message, multiply the quantity by the price and store it in `totalPrice`.
- Write the result into the total paragraph as Total: ₹ followed by `totalPrice.toFixed(2)`.
- Attach `calculateTotal()` to the input event of both the quantity field and the price field using `addEventListener()`.
- Save both files, open `index.html` in a browser and test the page with valid, zero, negative and empty values.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Product Calculator</title>

<style>
  body {
    font-family: Arial, sans-serif;
    margin: 40px;
  }

  .container {
    width: 350px;
    margin: auto;
    padding: 20px;
    border: 1px solid #ccc;
    border-radius: 10px;
  }

  label {
    display: block;
    margin-top: 10px;
  }

  input {
    width: 100%;
    padding: 8px;
    margin-top: 5px;
    box-sizing: border-box;
  }

  #total {
    margin-top: 20px;
    font-size: 20px;
    font-weight: bold;
  }

  #error {
    color: red;
    margin-top: 10px;
  }
</style>
</head>

<body>

  <div class="container">
    <h2>Product Calculator</h2>

    <label for="productName">Product Name:</label>
    <input type="text" id="productName" placeholder="Enter product
name">

    <label for="quantity">Quantity:</label>
    <input type="number" id="quantity" min="1" value="1">

    <label for="price">Price:</label>
```

```
        <input type="number" id="price" min="0" step="0.01"
placeholder="Enter price">
```

```
        <p id="error"></p>
        <p id="total">Total: ₹0.00</p>
    </div>
```

```
    <script src="script.js"></script>
</body>
</html>
```

script.js

```
// Get HTML elements using their IDs
const productName = document.getElementById("productName");
const quantity = document.getElementById("quantity");
const price = document.getElementById("price");
const total = document.getElementById("total");
const error = document.getElementById("error");

// Function to calculate total price
function calculateTotal() {
    const quantityValue = Number(quantity.value);
    const priceValue = Number(price.value);

    if (quantityValue <= 0 || priceValue < 0 || price.value === "") {
        error.textContent = "Please enter a valid quantity and price.";
        total.textContent = "Total: ₹0.00";
        return;
    }

    error.textContent = "";

    const totalPrice = quantityValue * priceValue;

    total.textContent = "Total: ₹" + totalPrice.toFixed(2);
}

// Add event listeners
quantity.addEventListener("input", calculateTotal);
price.addEventListener("input", calculateTotal);
```

7. CODE EXPLANATION

Code	Explanation
document.getElementById()	Selects the product name, quantity, price, total and error elements from the page by their id attribute.
element.value	Reads the text currently typed into an input field as a string.
Number()	Converts the string read from the input field into a numeric value so that it can be multiplied.
addEventListener("input", fn)	Runs calculateTotal() every time the user types in or changes the quantity or price field.

Code	Explanation
<pre>if (quantityValue <= 0 priceValue < 0 price.value === "")</pre>	Validation condition that rejects a zero or negative quantity, a negative price and an empty price field.
<code>error.textContent</code>	Displays the red validation message, or clears it by setting it to an empty string.
<code>return</code>	Stops the function immediately when the input is invalid, so no total is calculated.
<code>quantityValue * priceValue</code>	Calculates the total price from the validated quantity and price.
<code>toFixed(2)</code>	Rounds the total to two decimal places so the amount is displayed in currency format.
<code>total.textContent</code>	Writes the calculated total back into the page through the DOM, without reloading it.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested with valid inputs as well as invalid inputs (empty, zero and negative values). The results were recorded below.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Quantity shows 1, price is empty, no error message and Total: ₹0.00 is displayed	Default state displayed correctly	Pass
TC-02	Product: Wireless Mouse, Quantity: 2, Price: 750	Total updates to ₹1500.00 without reloading the page	Total: ₹1500.00 displayed	Pass
TC-03	Change the quantity from 2 to 5 (price unchanged)	Total recalculates immediately to ₹3750.00	Total updated dynamically	Pass
TC-04	Clear the price field (invalid: empty)	Error message "Please enter a valid quantity and price." is shown and the total resets to ₹0.00	Error message displayed, total reset	Pass
TC-05	Quantity: 0, Price: 750 (invalid: zero quantity)	Error message is shown and no total is calculated	Error message displayed, total ₹0.00	Pass
TC-06	Quantity: -3, Price: 750 (invalid: negative quantity)	Error message is shown and no total is calculated	Error message displayed, total ₹0.00	Pass

TC-07	Quantity: 2, Price: -50 (invalid: negative price)	Error message is shown and no total is calculated	Error message displayed, total ₹0.00	Pass
TC-08	Product: USB Cable, Quantity: 3, Price: 99.99	Total is ₹299.97, rounded to two decimal places	Total: ₹299.97 displayed	Pass
TC-09	After the error, enter a valid price of 120 with quantity 3	Error message disappears and Total: ₹360.00 is displayed	Error cleared, total recalculated	Pass

9. OUTPUT

TC-01 – Page loaded with default values

Product Calculator
Product Name:

Quantity:

Price:

Total: ₹0.00

TC-02 – Valid quantity and price entered

Product Calculator
Product Name:

Quantity:

Price:

Total: ₹1500.00

TC-03 – Quantity changed, total recalculated without reload

Product Calculator

Product Name:

Wireless Mouse

Quantity:

5

Price:

750

Total: ₹3750.00**TC-04 – Invalid input: price field left empty****Product Calculator**

Product Name:

Wireless Mouse

Quantity:

5

Price:

Enter price

Please enter a valid quantity and price.**Total: ₹0.00****TC-05 – Invalid input: quantity is zero**

Product Calculator

Product Name:

Wireless Mouse

Quantity:

0

Price:

750

Please enter a valid quantity and price.

Total: ₹0.00

TC-06 – Invalid input: negative quantity

Product Calculator

Product Name:

Wireless Mouse

Quantity:

-3

Price:

750

Please enter a valid quantity and price.

Total: ₹0.00

TC-07 – Invalid input: negative price

Product Calculator

Product Name:
Wireless Mouse

Quantity:
2

Price:
-50

Please enter a valid quantity and price.

Total: ₹0.00

TC-08 – Decimal price rounded to two decimal places

Product Calculator

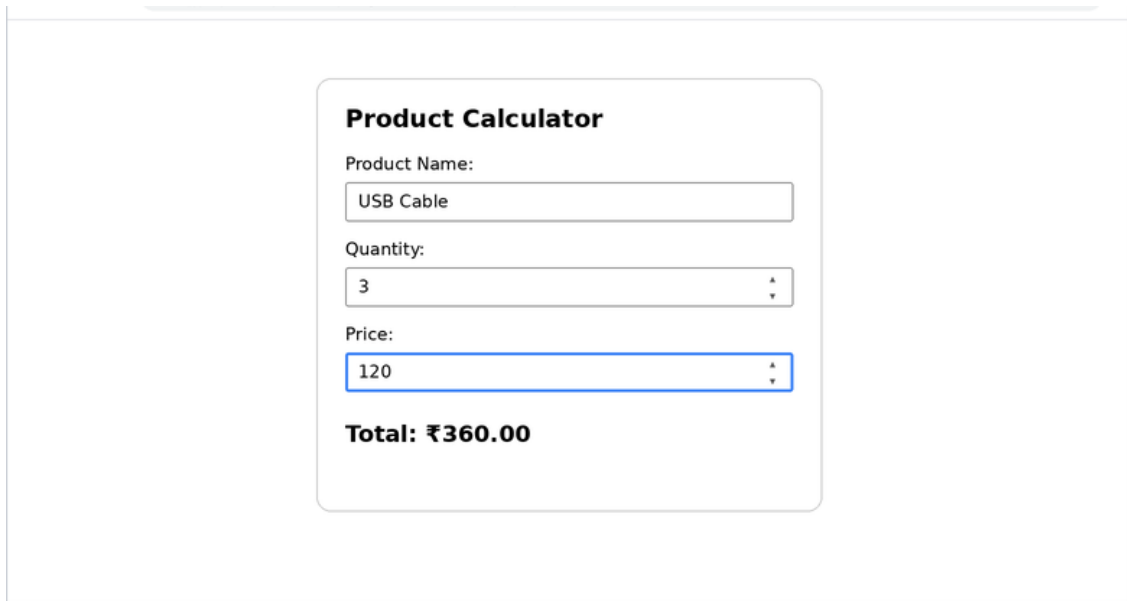
Product Name:
USB Cable

Quantity:
3

Price:
99.99

Total: ₹299.97

TC-09 – Valid value re-entered, error message cleared



The image shows a web browser window displaying a 'Product Calculator' form. The form is titled 'Product Calculator' in bold. It contains three input fields: 'Product Name:' with the value 'USB Cable', 'Quantity:' with the value '3', and 'Price:' with the value '120'. The 'Quantity' and 'Price' fields are spinners. Below the inputs, the total is displayed as 'Total: ₹360.00'.

Field	Value
Product Name	USB Cable
Quantity	3
Price	120
Total	₹360.00

10. RESULT

Thus, an interactive product calculator was successfully built using JavaScript functions, event listeners and DOM updates. The total price was calculated and displayed dynamically whenever the quantity or price changed, and the page was never reloaded. Invalid inputs such as an empty price, a zero quantity and negative values were rejected with a clear red error message, and the total was reset to ₹0.00 in those cases. One minor limitation was observed: the product name field is not validated, so a total is still calculated when the product name is left blank; adding a check on `productName.value.trim()` would make the validation complete.

EXPERIMENT 2

1. TITLE

Node.js File Management Application Using the File System (fs) Module.

2. AIM

To create a Node.js application that accepts a filename and content from the user and performs create/write, read, append and final display operations on that file using the File System module.

3. OBJECTIVE

The objectives of this experiment are:

- To take input from the user through the terminal using the built-in readline module.
- To validate the filename and display an error message when it is left empty.
- To create and write a file using `fs.writeFile()`.
- To read the contents of a file using `fs.readFile()` with utf8 encoding.
- To add further content to an existing file using `fs.appendFile()`.
- To display the final contents of the file after the append operation.
- To organise the file operations in a separate module and export them with `module.exports`.
- To handle errors such as an invalid path using the error object supplied to the callback.

4. THEORY

Node.js allows JavaScript to run outside the browser, on the server side. The File System module, imported with `require("fs")`, gives JavaScript direct access to files on the machine, which is not possible in a browser.

The asynchronous versions of the fs methods are used here. An asynchronous method does not stop the program while the disk operation takes place; instead, it accepts a callback function that Node.js runs once the operation has finished. The first argument passed to that callback is the error object, which is null when the operation succeeded. This is known as the error-first callback pattern, and it is the reason every callback in this program begins by checking whether an error occurred.

Because each step depends on the previous one, the callbacks are nested: the file is read only after it has been written, the content is appended only after the first read, and the final read happens only after the append. The readline module is used to ask the user for the filename and the content, since a terminal application has no HTML form to read values from.

The program is split into two files. `fileOperations.js` contains three wrapper functions, `writeFile()`, `readFile()` and `appendFile()`, which are exported with `module.exports`. `app.js` imports them with `require("./fileOperations")` and uses them in order. Keeping the file operations in their own module makes the main program easier to read and allows the same functions to be reused elsewhere.

The key methods and concepts used in this program are:

- `require("fs")` imports the built-in File System module.
- `require("readline")` imports the module used to read input typed in the terminal.
- `readline.createInterface({ input: process.stdin, output: process.stdout })` connects the program to the keyboard and the screen.
- `rl.question(prompt, callback)` prints a prompt and passes whatever the user types to the callback.
- `fs.writeFile(filename, content, "utf8", callback)` creates the file, or replaces its contents if the file already exists.

- `fs.readFile(filename, "utf8", callback)` reads the file and returns its contents as a string; without "utf8" a Buffer would be returned instead.
- `fs.appendFile(filename, text, "utf8", callback)` adds text to the end of the file without deleting what is already there; a "\n" is added first so the new content starts on a new line.
- `module.exports` exposes the three functions so that `app.js` can import them.
- `filename.trim() === ""` treats a filename made only of spaces as empty.
- `error.message` prints a readable description of a failure, such as `ENOENT` when the folder does not exist.
- `rl.close()` ends the readline interface so that the program exits.

The basic flow of the program is:

The program asks for a filename → the filename is validated → the program asks for the initial content → the file is created and written → the file is read and its contents are printed → the program asks for additional content → that content is appended → the file is read once more and the final contents are printed → the readline interface is closed.

5. ALGORITHM / PROCEDURE

- Create a project folder and initialise it with `npm init -y`, then create two files, `fileOperations.js` and `app.js`.
- In `fileOperations.js`, import the `fs` module with `require("fs")`.
- Define `writeFile(filename, content, callback)` that calls `fs.writeFile()` and passes the error, if any, to the callback.
- Define `readFile(filename, callback)` that calls `fs.readFile()` with `utf8` encoding and passes the error and the data to the callback.
- Define `appendFile(filename, additionalContent, callback)` that calls `fs.appendFile()` with a newline before the new content.
- Export all three functions using `module.exports`.
- In `app.js`, import `readline` and the three functions from `./fileOperations`.
- Create the readline interface using `process.stdin` and `process.stdout`.
- Ask the user for the filename, and if the trimmed filename is empty, print "Filename cannot be empty.", close the interface and stop.
- Ask the user for the initial content and call `writeFile()`; if an error is returned, print the error message and stop.
- On success, print a confirmation and call `readFile()` to display the initial contents of the file.
- Ask the user for additional content and call `appendFile()`; print a confirmation when it succeeds.
- Call `readFile()` once more to print the final contents of the file, then close the readline interface.
- Run the program with `node app.js` and test it with a valid filename, an empty filename, a filename in a folder that does not exist and an existing file.

6. PROGRAM

fileOperations.js

```
const fs = require("fs");

// Function to write/create a file
function writeFile(filename, content, callback) {
  fs.writeFile(filename, content, "utf8", (error) => {
    if (error) {
      callback(error);
    }
    return;
  });
}
```

```

        }

        callback(null);
    });
}

// Function to read a file
function readFile(filename, callback) {
    fs.readFile(filename, "utf8", (error, data) => {
        if (error) {
            callback(error, null);
            return;
        }

        callback(null, data);
    });
}

// Function to append content to a file
function appendFile(filename, additionalContent, callback) {
    fs.appendFile(
        filename,
        "\n" + additionalContent,
        "utf8",
        (error) => {
            if (error) {
                callback(error);
                return;
            }

            callback(null);
        }
    );
}

// Export all functions
module.exports = {
    writeFile,
    readFile,
    appendFile
};

```

app.js

```

const readline = require("readline");

const {
    writeFile,
    readFile,
    appendFile
} = require("../fileOperations");

// Create interface for user input
const rl = readline.createInterface({
    input: process.stdin,

```

```

        output: process.stdout
    });

    // Ask for filename
    rl.question("Enter filename: ", (filename) => {

        // Validate filename
        if (filename.trim() === "") {
            console.log("Filename cannot be empty.");
            rl.close();
            return;
        }

        // Ask for initial content
        rl.question("Enter initial content: ", (content) => {

            // Create/write the file
            writeFile(filename, content, (writeError) => {

                if (writeError) {
                    console.log(
                        "Error while writing file:",
                        writeError.message
                    );
                    rl.close();
                    return;
                }

                console.log("File created/written successfully.");

                // Read the file
                readFile(filename, (readError, data) => {

                    if (readError) {
                        console.log(
                            "Error while reading file:",
                            readError.message
                        );
                        rl.close();
                        return;
                    }

                    console.log("\nInitial file contents:");
                    console.log(data);

                    // Ask for additional content
                    rl.question(
                        "\nEnter additional content to append: ",
                        (additionalContent) => {

                            // Append content
                            appendFile(
                                filename,
                                additionalContent,

```



```

    "keywords": [],
    "author": "",
    "license": "ISC",
    "type": "commonjs"
  }

```

7. CODE EXPLANATION

Code	Explanation
<code>require("fs")</code>	Imports the built-in File System module that provides the file operations.
<code>require("readline")</code>	Imports the module used to read the filename and the content typed by the user.
<code>readline.createInterface()</code>	Connects the program to the keyboard (process.stdin) and the screen (process.stdout).
<code>rl.question(prompt, cb)</code>	Displays a prompt in the terminal and passes the user's answer to the callback.
<code>filename.trim() === ""</code>	Validation check that treats a blank or spaces-only filename as empty.
<code>fs.writeFile()</code>	Creates the file with the given content, or overwrites it if the file already exists.
<code>fs.readFile(filename, "utf8", cb)</code>	Reads the file and returns its contents as a readable string instead of a Buffer.
<code>fs.appendFile()</code>	Adds the additional content to the end of the file on a new line, keeping the existing content.
<code>callback(error) / callback(null, data)</code>	The error-first callback pattern; the first argument is null when the operation succeeded.
<code>error.message</code>	Prints a readable description of the failure, such as a missing folder.
<code>module.exports</code>	Exports <code>writeFile</code> , <code>readFile</code> and <code>appendFile</code> so that <code>app.js</code> can import them.
<code>rl.close()</code>	Closes the readline interface so the program ends.

8. TEST CASES AND EXECUTION DATA

The program was run from the terminal using `node app.js` and tested with valid input as well as invalid input such as an empty filename and a path that does not exist.

Test Case	Action / Input	Expected Result	Actual Result	Status

TC-01	Filename: notes.txt, content: "This is the first line of the file.", append: "This line was appended later."	File is created, read, appended to, and the final contents show both lines	All four operations completed in order	Pass
-------	--	--	--	------

TC-02	Press Enter without typing a filename (invalid: empty)	The message "Filename cannot be empty." is displayed and the program stops	Error message displayed, program exited	Pass
TC-03	Filename entered as spaces only (invalid)	trim() treats it as empty and the same error message is shown	Error message displayed, program exited	Pass
TC-04	Filename: nofolder/data.txt (invalid: folder does not exist)	An ENOENT write error message is displayed and no file is created	Error while writing file: ENOENT ... displayed	Pass
TC-05	Re-run with the existing filename notes.txt and new content	The old content is replaced by the new content, then the appended line is added	File overwritten and appended correctly	Pass
TC-06	Filename: empty.txt with the content left blank	An empty file is created and the appended line becomes the only content	Empty file created, append worked	Pass

9. OUTPUT

TC-01 – Valid filename and content: create, read, append and final display

```

PS C:\Users\vanis\Desktop\FS_LAB\task-5\task-5.2> node app.js
Enter filename: notes.txt
Enter initial content: hello
File created/written successfully.

Initial file contents:
hello

Enter additional content to append: nothing
Content appended successfully.

Final file contents:
hello
nothing

```

TC-02 – Invalid input: filename left empty

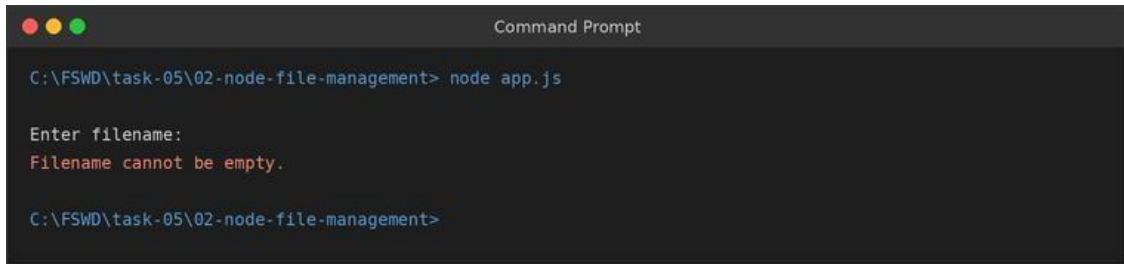
```
C:\FSWD\task-05\02-node-file-management> node app.js
```

```
Enter filename:
```

```
Filename cannot be empty.
```

```
C:\FSWD\task-05\02-node-file-management>
```

TC-03 – Invalid input: filename containing only spaces



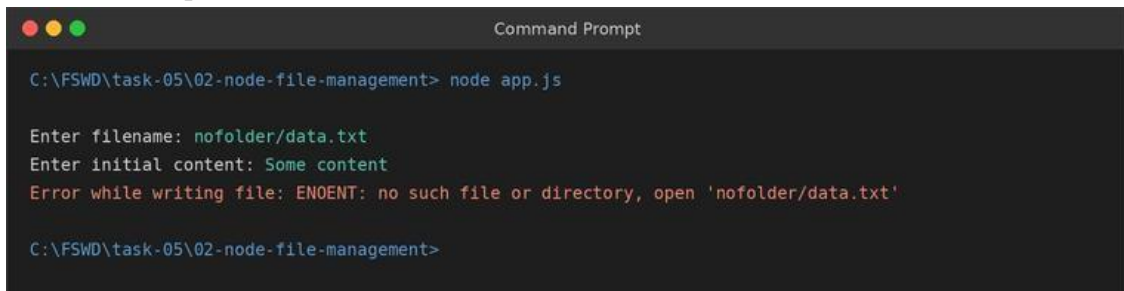
```
Command Prompt

C:\FSWD\task-05\02-node-file-management> node app.js

Enter filename:
Filename cannot be empty.

C:\FSWD\task-05\02-node-file-management>
```

TC-04 – Invalid input: filename in a folder that does not exist



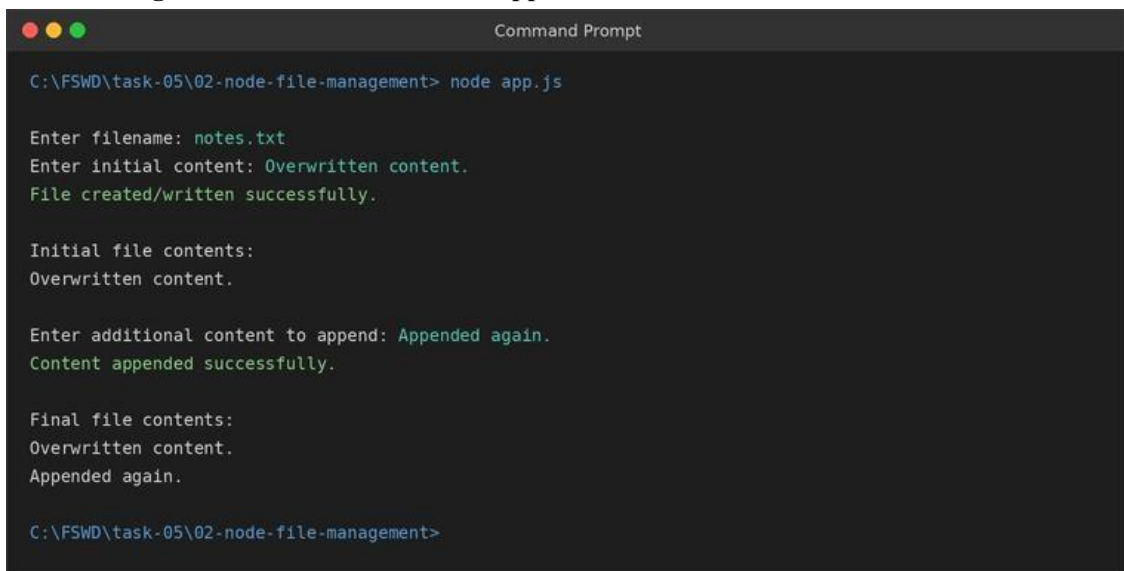
```
Command Prompt

C:\FSWD\task-05\02-node-file-management> node app.js

Enter filename: nofolder/data.txt
Enter initial content: Some content
Error while writing file: ENOENT: no such file or directory, open 'nofolder/data.txt'

C:\FSWD\task-05\02-node-file-management>
```

TC-05 – Existing file is overwritten and then appended to



```
Command Prompt

C:\FSWD\task-05\02-node-file-management> node app.js

Enter filename: notes.txt
Enter initial content: Overwritten content.
File created/written successfully.

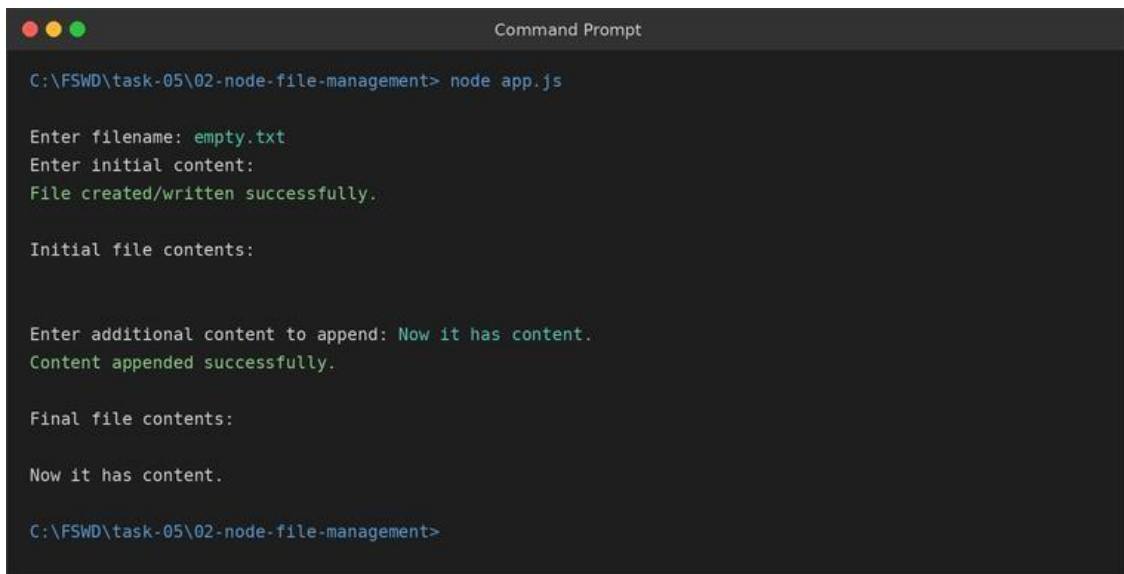
Initial file contents:
Overwritten content.

Enter additional content to append: Appended again.
Content appended successfully.

Final file contents:
Overwritten content.
Appended again.

C:\FSWD\task-05\02-node-file-management>
```

TC-06 – Empty initial content accepted, append adds the first line



```
C:\FSWD\task-05\02-node-file-management> node app.js

Enter filename: empty.txt
Enter initial content:
File created/written successfully.

Initial file contents:

Enter additional content to append: Now it has content.
Content appended successfully.

Final file contents:

Now it has content.

C:\FSWD\task-05\02-node-file-management>
```

10. RESULT

Thus, a Node.js file management application was successfully created using the File System module. The program accepted a filename and content from the user, created and wrote the file, read and displayed its contents, appended additional content and displayed the final contents. Invalid input was handled correctly: an empty or spaces-only filename produced a clear error message, and a path that does not exist produced an ENOENT write error instead of crashing the program. One point noted during testing is that `writeFile()` silently replaces an existing file, so running the program twice with the same filename loses the earlier content; asking the user for confirmation before overwriting would make the application safer.

EXPERIMENT 3

1. TITLE

Express.js Student Server with Routing and HTTP Methods.

2. AIM

To create a basic Express.js server with the routes `/`, `/students` and `/about`, where `/students` returns a list of at least five students and `/about` displays information about the application, using appropriate HTTP methods and routing.

3. OBJECTIVE

The objectives of this experiment are:

- To install Express.js in a Node.js project using npm and import it with `require("express")`.
- To create an Express application object and start a server on a chosen port with `app.listen()`.
- To define GET routes for `/`, `/students` and `/about` using `app.get()`.
- To send a plain text response using `res.send()`.
- To send a JSON response containing an array of five student objects using `res.json()`.
- To handle requests to routes that do not exist by sending a 404 response.
- To set an HTTP status code explicitly using `res.status()`.
- To verify each route in the browser and in Postman.

4. THEORY

Express.js is a web application framework built on top of the Node.js http module. It removes the repetitive work of parsing URLs and methods by hand and lets the developer describe the server as a set of routes. A route is a combination of an HTTP method and a path; when a request arrives, Express matches it against the routes in the order they were defined and runs the first handler that matches.

Every route handler receives two objects. The request object, `req`, describes what the client asked for, and the response object, `res`, is used to send the answer back. A response must be sent exactly once per request, which is why each handler here ends with a single `res.send()` or `res.json()` call.

`res.send()` sends text or HTML and sets the Content-Type header to `text/html`, while `res.json()` converts a JavaScript array or object into JSON text and sets the Content-Type header to `application/json`. Because the student data is an array of objects, `res.json()` is the correct choice for the `/students` route.

The `app.use()` call placed after all the routes acts as a catch-all. Express reaches it only when no earlier route has matched the request, so it is the natural place to send the 404 response. Its position in the file matters: if it were written before the routes, every request would return 404.

The key methods and concepts used in this program are:

- `require("express")` imports the Express framework, which must first be installed with `npm install express`.
- `const app = express()` creates the application object that holds all the routes.
- `app.get(path, handler)` registers a handler for GET requests to that path, which is the method a browser uses when a URL is typed into the address bar.
- `res.send(text)` sends a plain text or HTML response to the client.
- `res.json(array)` sends the student array as JSON, setting the correct content type automatically.

- `app.use(handler)` placed after all routes runs only when nothing else matched, and is used here as the 404 handler.
- `res.status(404)` sets the HTTP status code so that the client knows the resource was not found.
- `app.listen(PORT, callback)` starts the server and runs the callback once it is listening, printing the address to the terminal.
- A template literal written with backticks is used to build the startup message with the port number inside it.

The basic flow of the program is:

The server is started with `node index.js` → Express listens on port 3000 → a browser or Postman sends a GET request → Express compares the path with the registered routes → `/` returns the welcome text, `/students` returns the JSON array, `/about` returns the description → any other path falls through to `app.use()` and returns 404 - Page not found.

5. ALGORITHM / PROCEDURE

- Create a project folder and run `npm init -y` to generate `package.json`.
- Install Express with `npm install express`.
- Create `index.js` and import Express with `require("express")`.
- Create the application object with `express()` and store the port number 3000 in a constant.
- Define an array named `students` containing five objects, each with `id`, `name` and `course`.
- Define a GET route for `/` that sends the welcome message using `res.send()`.
- Define a GET route for `/students` that sends the students array using `res.json()`.
- Define a GET route for `/about` that sends a description of the application using `res.send()`.
- After all the routes, add `app.use()` that sets the status to 404 and sends "404 - Page not found".
- Call `app.listen(PORT, callback)` and print the server address from the callback.
- Start the server with `node index.js` and confirm the startup message in the terminal.
- Open `http://localhost:3000/`, `/students` and `/about` in a browser, test an undefined route, and verify `/students` in Postman.

6. PROGRAM

`index.js`

```
const express = require("express");

const app = express();

const PORT = 3000;

// Student data
const students = [
  { id: 1, name: "Anjali", course: "CSE" },
  { id: 2, name: "Rahul", course: "ECE" },
  { id: 3, name: "Priya", course: "CSE" },
  { id: 4, name: "Arjun", course: "IT" },
  { id: 5, name: "Sneha", course: "AI & ML" }
];

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to the Student Management Server");
});
```

```

});

// Students route
app.get("/students", (req, res) => {
  res.json(students);
});

// About route
app.get("/about", (req, res) => {
  res.send(
    "This is a basic Express.js Student Management Application."
  );
});

// Handle unknown routes
app.use((req, res) => {
  res.status(404).send("404 - Page not found");
});

// Start server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});

```

package.json

```

{
  "name": "03-student_server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}

```

7. CODE EXPLANATION

Code	Explanation
<code>require("express")</code>	Imports the Express framework installed through npm.
<code>const app = express()</code>	Creates the Express application object on which the routes are registered.
<code>const PORT = 3000</code>	Stores the port number the server listens on.
<code>const students = [...]</code>	An array of five student objects, each with an id, a name and a course.

Code	Explanation
<code>app.get("/", handler)</code>	Handles GET requests to the home route and sends the welcome message.
<code>app.get("/students", handler)</code>	Handles GET requests to /students and returns the student list.
<code>res.json(students)</code>	Converts the array into JSON and sets the Content-Type header to application/json.
<code>app.get("/about", handler)</code>	Handles GET requests to /about and sends information about the application.
<code>res.send(text)</code>	Sends a plain text response to the browser or to Postman.
<code>app.use((req, res) => ...)</code>	Catch-all handler placed after all routes; it runs only when no route matched.
<code>res.status(404)</code>	Sets the HTTP status code to 404 before sending the not-found message.
<code>app.listen(PORT, callback)</code>	Starts the server and prints the address once it is listening.

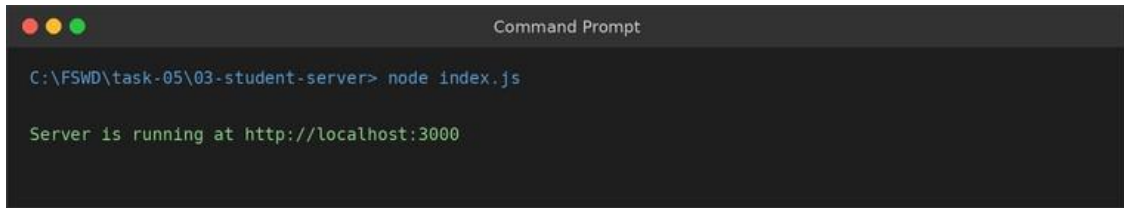
8. TEST CASES AND EXECUTION DATA

The server was started from the terminal and each route was verified in the browser and in Postman, including a route that does not exist and an HTTP method that is not defined.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Run node index.js in the terminal	The message "Server is running at http://localhost:3000" is printed	Server started on port 3000	Pass
TC-02	Open http://localhost:3000/ in a browser	The text "Welcome to the Student Management Server" is displayed	Welcome message displayed	Pass
TC-03	Open http://localhost:3000/students	A JSON array of five students with id, name and course is returned	JSON list of five students displayed	Pass
TC-04	Open http://localhost:3000/about	The application description is displayed	About text displayed	Pass
TC-05	Open http://localhost:3000/teachers (invalid route)	The catch-all handler returns "404 - Page not found" with status 404	404 message displayed	Pass
TC-06	Send GET http://localhost:3000/students from Postman	Status 200 OK with the student list in JSON format	200 OK, JSON body received	Pass
TC-07	Send POST http://localhost:3000/students from Postman (method not defined)	The request does not match any route and returns 404 - Page not found	404 Not Found returned	Pass

9. OUTPUT

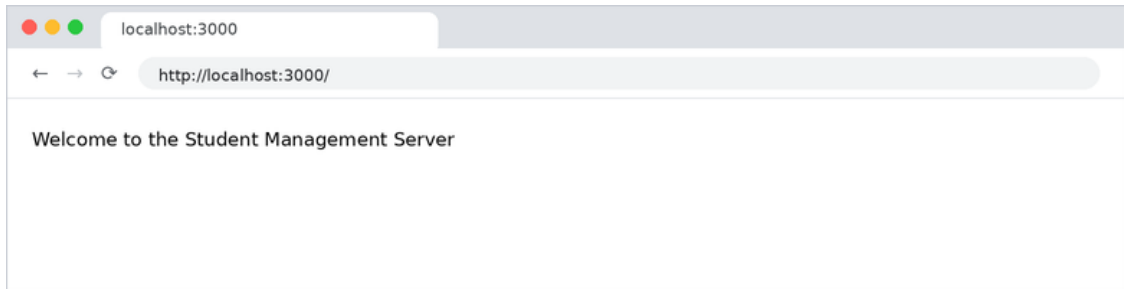
TC-01 – Server started from the terminal



```
Command Prompt
C:\FSWD\task-05\03-student-server> node index.js

Server is running at http://localhost:3000
```

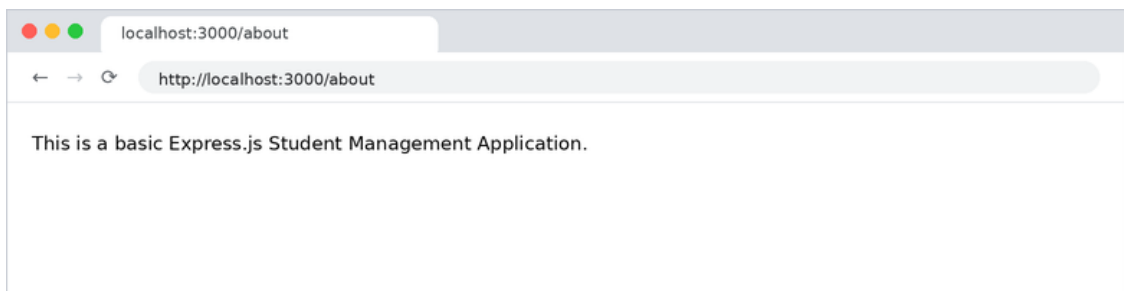
TC-02 – Home route / in the browser



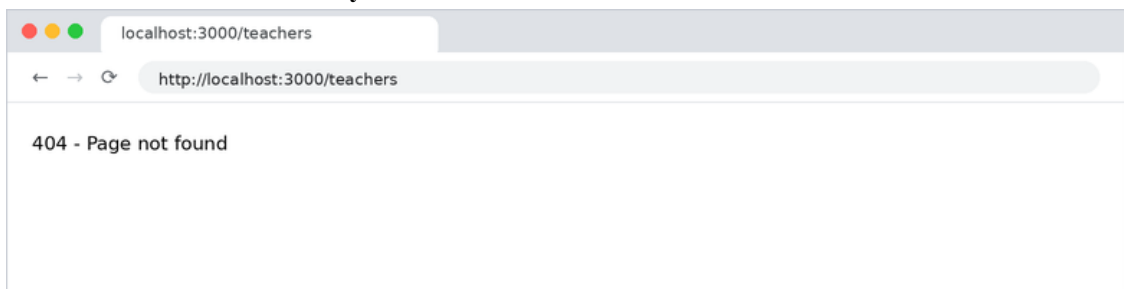
TC-03 – /students route returning the JSON student list



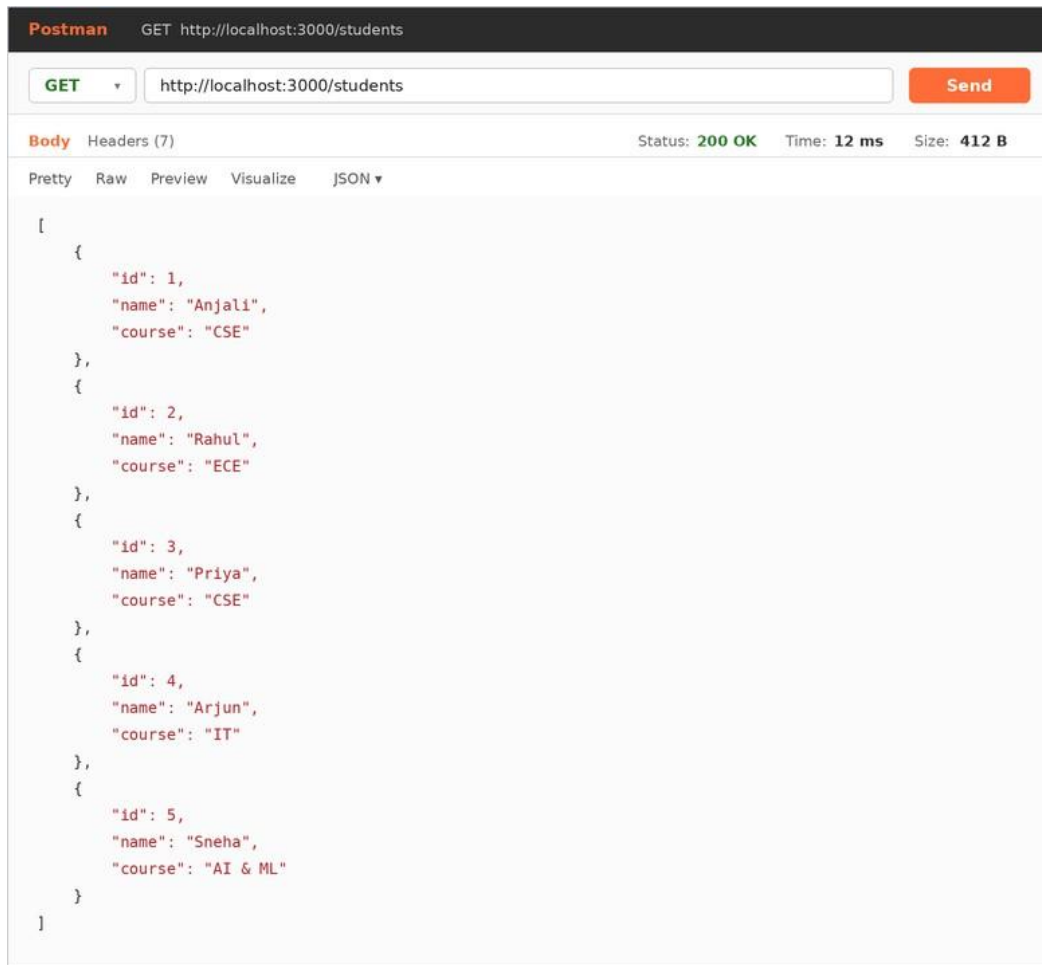
TC-04 – /about route in the browser



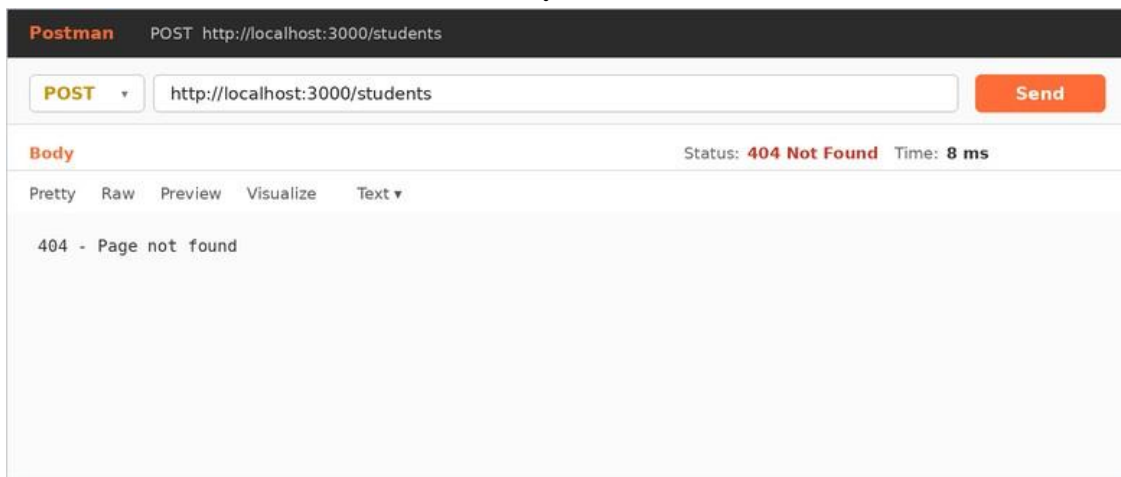
TC-05 – Invalid route handled by the 404 handler



TC-06 – GET /students verified in Postman (200 OK)



TC-07 – POST /students returns 404 because only GET is defined



10. RESULT

Thus, a basic Express.js student server was successfully created with the routes /, /students and /about. The home and about routes returned plain text using `res.send()`, the students route returned a JSON array of five students using `res.json()`, and all of them were verified in the browser and in Postman. Requests to an undefined route, and requests using an HTTP method that is not defined, were handled by the catch-all middleware and returned 404 - Page not found with the correct status code. A possible improvement is to add POST, PUT and DELETE routes so that students can also be added, updated and removed rather than only listed.

EXPERIMENT 4

1. TITLE

Demonstration of the Document Object Model (DOM): Changing the Content, Style and Attributes of an HTML Element Using JavaScript.

2. AIM

To explain the Document Object Model and to demonstrate how JavaScript can dynamically change the content, the style and the attributes of an HTML element at runtime.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the structure of the Document Object Model and how the browser represents an HTML page as a tree of nodes.
- To access an element on the page from JavaScript using `document.getElementById()`.
- To change the visible text of an element using the `textContent` property.
- To change the appearance of an element at runtime using the style object.
- To add or modify an attribute of an element using `setAttribute()`.
- To run all of these changes from a single click event handler attached with `addEventListener()`.
- To verify the changes visually in the browser and in the browser developer tools.

4. THEORY

When a browser loads an HTML page, it does not keep the file as plain text. It parses the file and builds an object-based representation of it in memory called the Document Object Model, or DOM. In this model every element, every attribute and every piece of text becomes a node, and the nodes are arranged as a tree with the document object at the root, `<html>` below it, then `<head>` and `<body>`, and so on down to the individual elements.

The value of the DOM is that it is a live model. It is not a copy of the HTML file; it is what the browser actually displays. Whenever JavaScript changes a node in the DOM, the browser immediately repaints the page to match, which is how a page can change without being reloaded and without the original HTML file being edited in any way. Reloading the page rebuilds the DOM from the file, so all such changes are lost.

JavaScript can change three different things about an element, and this program demonstrates all three:

- Content – the text inside the element, changed here with the `textContent` property, which replaces whatever text the element contained.
- Style – the appearance of the element, changed here through the style object; `heading.style.color` sets an inline CSS colour, and CSS properties written with a hyphen such as `background-color` become camelCase in JavaScript, so it is written as `backgroundColor`.
- Attributes – the extra information carried by the tag, changed here with `setAttribute("title", ...)`, which adds a title attribute that the browser then shows as a tooltip when the mouse rests on the heading.

The changes are made inside a function passed to `addEventListener("click", ...)`, so nothing happens until the user clicks the button. This is the event-driven model on which interactive web pages are built.

The key methods and concepts used in this program are:

- `document.getElementById("id")` returns the node in the DOM tree whose id attribute matches.

- `addEventListener("click", fn)` registers a function to run when the button is clicked.
- `element.textContent` replaces the text inside an element; unlike `innerHTML`, it treats the value as plain text and does not interpret HTML tags, which makes it the safer choice.
- `element.style.color` and `element.style.backgroundColor` set inline CSS on the element, which overrides the styles defined in a stylesheet.
- `element.style.padding` adds inline spacing inside the element.
- `element.setAttribute("title", value)` adds or updates an attribute on the element; the title attribute is displayed by the browser as a tooltip.
- The changes exist only in the DOM in memory, so refreshing the page restores the original HTML.

The basic flow of the page is:

The page loads and the browser builds the DOM → the three elements are selected and stored in constants → the user clicks the Change Content button → the click handler runs → the heading text and the paragraph text are replaced → the heading colour, background colour and padding are set → a title attribute is added to the heading → the browser repaints the page immediately, without reloading it.

5. ALGORITHM / PROCEDURE

- Create an HTML file containing an `<h1>` with `id="heading"`, a `<p>` with `id="message"` and a `<button>` with `id="changeButton"`.
- Write the JavaScript inside a `<script>` tag placed at the end of the body, so the elements already exist in the DOM when the script runs.
- Select the heading, the paragraph and the button using `document.getElementById()` and store them in constants.
- Attach a click event listener to the button using `addEventListener()`.
- Inside the handler, change the content by setting `heading.textContent` and `message.textContent` to their new values.
- Change the style by setting `heading.style.color` to blue, `heading.style.backgroundColor` to lightgray and `heading.style.padding` to 10px.
- Change the attribute by calling `heading.setAttribute("title", "This is a modified heading")`.
- Save the file and open it in a browser; note the original content, then click the button and observe the content, style and attribute change.
- Hover the mouse over the heading to confirm that the tooltip from the new title attribute appears.
- Open the browser developer tools and inspect the heading to confirm the inline style and the title attribute in the DOM.
- Refresh the page to confirm that the DOM is rebuilt from the HTML file and the original content returns.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Demonstration</title>
</head>

<body>
```

```

<h1 id="heading">Welcome to DOM</h1>

<p id="message">
  This is the original paragraph.
</p>

<button id="changeButton">
  Change Content
</button>

<script>
  const heading = document.getElementById("heading");
  const message = document.getElementById("message");
  const button = document.getElementById("changeButton");

  button.addEventListener("click", function () {

    // Change HTML content
    heading.textContent = "DOM Content Changed";

    message.textContent =
      "JavaScript dynamically modified this paragraph.";

    // Change style
    heading.style.color = "blue";
    heading.style.backgroundColor = "lightgray";
    heading.style.padding = "10px";

    // Change an attribute
    heading.setAttribute("title", "This is a modified heading");
  });

</script>

</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById("heading")</code>	Selects the <code><h1></code> node from the DOM so that JavaScript can modify it.
<code>document.getElementById("message")</code>	Selects the paragraph whose text will be replaced.
<code>document.getElementById("changeButton")</code>	Selects the button that triggers all the changes.
<code>addEventListener("click", function () { ... })</code>	Runs the given function whenever the button is clicked, instead of when the page loads.

<code>heading.textContent = "DOM Content Changed"</code>	Changes the content: replaces the text inside the heading.
--	--

Code	Explanation
<code>message.textContent = "..."</code>	Changes the content of the paragraph in the same way.
<code>heading.style.color = "blue"</code>	Changes the style: applies an inline CSS colour to the heading.
<code>heading.style.backgroundColor = "lightgray"</code>	Applies a background colour; the hyphenated CSS property background-color becomes backgroundColor in JavaScript.
<code>heading.style.padding = "10px"</code>	Adds inline spacing inside the heading element.
<code>heading.setAttribute("title", "...")</code>	Changes the attributes: adds a title attribute, which the browser displays as a tooltip on hover.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested before and after the click, including a check in the developer tools and a check that the change is not permanent.

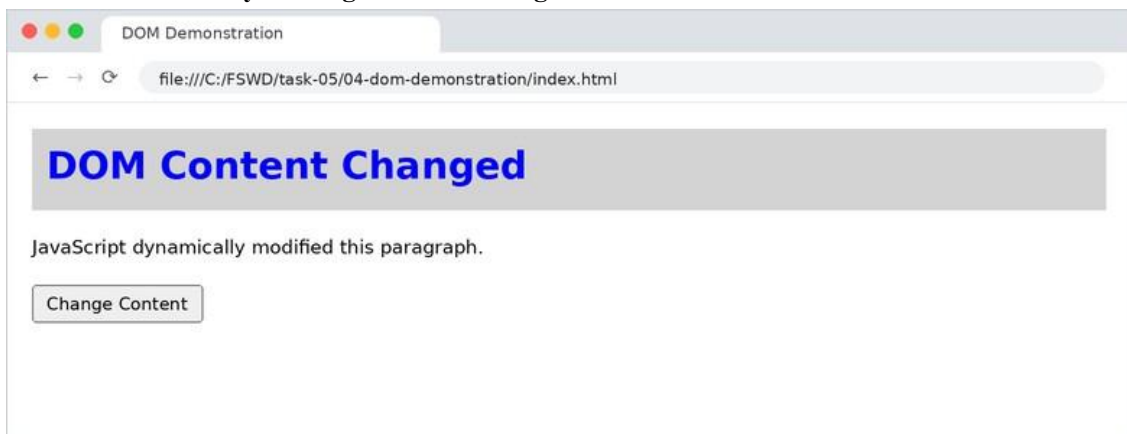
Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	The original heading and paragraph are displayed in the default black text with no background	Original content displayed	Pass
TC-02	Click the "Change Content" button	The heading and the paragraph text change, the heading turns blue with a light grey background and padding	Content and style changed	Pass
TC-03	Hover the mouse over the changed heading	The tooltip "This is a modified heading" appears from the new title attribute	Tooltip displayed on hover	Pass
TC-04	Inspect the heading in the browser developer tools	The inline style and the title attribute are visible on the <h1> element in the DOM	Style and attribute present in the DOM	Pass
TC-05	Click the button a second time	The same values are set again, so the page stays as it is with no error	Output unchanged, no error	Pass
TC-06	Refresh the page after clicking	The DOM is rebuilt from the HTML file and the original content returns	Original content restored	Pass

9. OUTPUT

TC-01 – Original page before the button is clicked



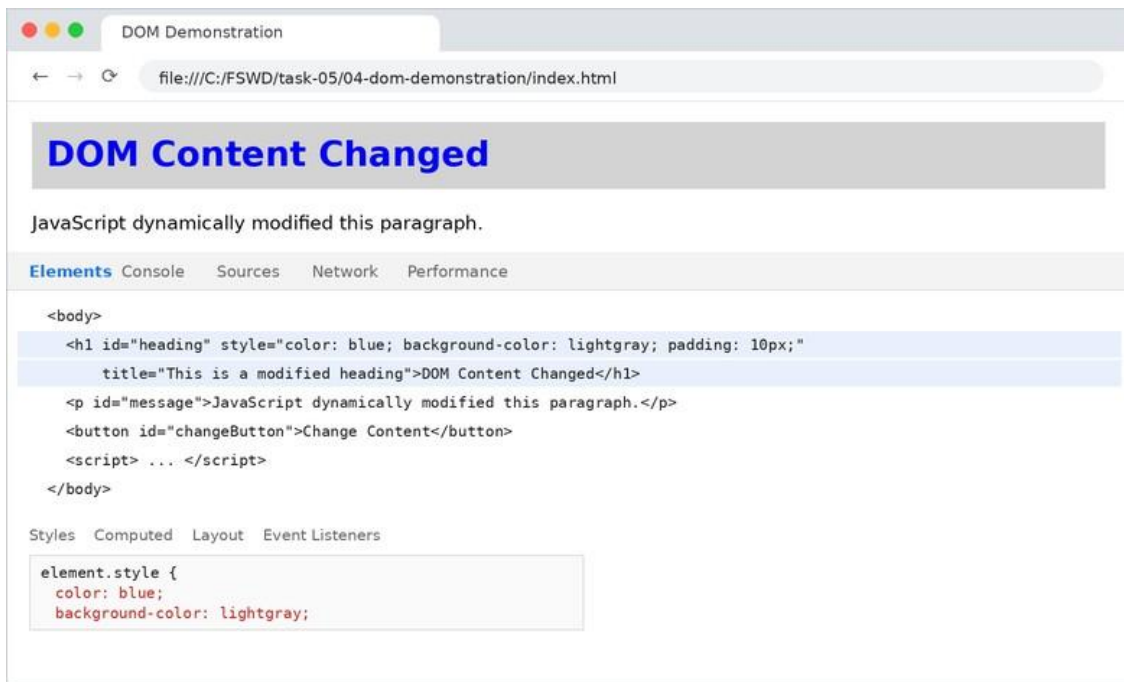
TC-02 – Content and style changed after clicking the button



TC-03 – Tooltip from the newly added title attribute



TC-04 – Modified style and attribute seen in the developer tools



TC-05 – Clicking a second time leaves the output unchanged



TC-06 – After refreshing, the original DOM is rebuilt from the HTML file



10. RESULT

Thus, the Document Object Model was explained and demonstrated successfully. On clicking the button, JavaScript changed the content of the heading and the paragraph using `textContent`, changed

the style of the heading using the style object, and changed its attributes using `setAttribute()`, and the browser repainted the page immediately without reloading it. The developer tools confirmed that the inline style and the title attribute had been added to the element in the DOM. It was also observed that refreshing the page restores the original content, because the DOM is rebuilt from the HTML file each time the page is loaded and the changes are only held in memory.